

Факультет программной инженерии и компьютерной техники

Основы программной инженерии

Лабораторная работа №4

Вариант 666999

Выполнил:

Евграфов Артём Андреевич

Группа Р3209

Проверил:

Преподаватель основ программной инженерии

Ермаков Михаил Константинович

Содержание

Текст задания	2
Реализация	3
1. Исходный код MBean	3
2. JConsole	7
3. VisualVM	12
4. Исследование программы на утечки памяти	16
Вывод	21

Текст задания

Для программы из лабораторной работы №3 по дисциплине «Веб-программирование» необходимо выполнить следующие задания.

1. Реализовать:

- MBean, считающий общее число установленных пользователем точек и число точек, не попадающих в область. Если количество установленных пользователем точек стало кратно 10, MBean должен отправить оповещение об этом событии;
- MBean, определяющий площадь получившейся фигуры.

2. С помощью утилиты JConsole провести мониторинг программы:

- снять показания MBean-классов, разработанных при выполнении задания 1;
- определить время в миллисекундах, прошедшее с момента запуска виртуальной машины.

3. С помощью утилиты VisualVM провести мониторинг и профилирование программы:

- снять график изменения показаний разработанных MBean-классов с течением времени;
- определить имя класса, объекты которого занимают наибольший объём памяти JVM, и определить пользовательский класс, в экземплярах которого находятся эти объекты.

4. С помощью VisualVM и профилировщика IDE локализовать и устранить проблемы с производительностью в программе. В отчёте привести описание выявленной проблемы, способ её устранения и подробный алгоритм локализации со скриншотами. Обеспечить воспроизводимость процесса поиска проблемы.

Реализация

1. Исходный код MBean

В приложении реализованы два стандартных MBean. `PointStatistics` хранит число проверенных точек, попаданий и промахов и формирует JMX-уведомление после каждой десятой точки. `Area` возвращает текущий радиус и площадь заданной вариантном области. Оба MBean используют общее потоко-безопасное состояние и регистрируются в платформенном MBeanServer при развёртывании приложения.

- `mbean/PointStatisticsMBean.java`

```
package com.example.lab3.mbean;

public interface PointStatisticsMBean {

    long getTotalPoints();

    long getMissPoints();

    long getHitPoints();

    void reset();
}
```

- `mbean/PointStatistics.java`

```
package com.example.lab3.mbean;

import javax.management.MBeanNotificationInfo;
import javax.management.Notification;
import javax.management.NotificationBroadcasterSupport;
import java.util.concurrent.atomic.AtomicLong;

public class PointStatistics extends NotificationBroadcasterSupport implements PointStatisticsMBean {

    public static final String TENTH_POINT_NOTIFICATION = "com.example.lab4.point.totalMultipleOfTen";

    private final MonitoringState state;
    private final AtomicLong sequence = new AtomicLong();

    PointStatistics(MonitoringState state) {
        this.state = state;
    }

    void registerPoint(boolean hit, double radius) {
        MonitoringState.Snapshot snapshot = state.registerPoint(hit, radius);
        if (snapshot.tenthPoint()) {
            Notification notification = new Notification(
                TENTH_POINT_NOTIFICATION,
                this,
                sequence.incrementAndGet(),
                System.currentTimeMillis(),
                "Total number of user points became multiple of 10: " + snapshot.totalPoints()
            );
            notification.setUserData("total=" + snapshot.totalPoints()
                + ", hits=" + (snapshot.totalPoints() - snapshot.missPoints())
                + ", misses=" + snapshot.missPoints());
            sendNotification(notification);
        }
    }

    @Override
```

```

    public long getTotalPoints() {
        return state.snapshot().totalPoints();
    }

    @Override
    public long getMissPoints() {
        return state.snapshot().missPoints();
    }

    @Override
    public long getHitPoints() {
        MonitoringState.Snapshot snapshot = state.snapshot();
        return snapshot.totalPoints() - snapshot.missPoints();
    }

    @Override
    public void reset() {
        state.reset();
    }

    @Override
    public MBeanNotificationInfo[] getNotificationInfo() {
        String[] types = {TENTH_POINT_NOTIFICATION};
        String name = Notification.class.getName();
        String description = "Notification sent when total user point count is multiple of 10";
        return new MBeanNotificationInfo[]{new MBeanNotificationInfo(types, name, description)};
    }
}

```

- mbean/AreaMBean.java

```

package com.example.lab3.mbean;

public interface AreaMBean {

    double getRadius();

    double getArea();
}

```

- mbean/Area.java

```

package com.example.lab3.mbean;

public class Area implements AreaMBean {

    private final MonitoringState state;

    Area(MonitoringState state) {
        this.state = state;
    }

    @Override
    public double getRadius() {
        return state.snapshot().currentRadius();
    }

    @Override
    public double getArea() {
        return state.snapshot().area();
    }
}

```

- mbean/MonitoringState.java

```

package com.example.lab3.mbean;

final class MonitoringState {

    private long totalPoints;
    private long missPoints;
    private double currentRadius = 1.5;

    synchronized Snapshot registerPoint(boolean hit, double radius) {
        totalPoints++;
        currentRadius = radius;
        if (!hit) {
            missPoints++;
        }

        return snapshot(totalPoints % 10 == 0);
    }

    synchronized Snapshot snapshot() {
        return snapshot(false);
    }

    synchronized void updateRadius(double radius) {
        currentRadius = radius;
    }

    synchronized void reset() {
        totalPoints = 0;
        missPoints = 0;
    }

    private Snapshot snapshot(boolean tenthPoint) {
        return new Snapshot(totalPoints, missPoints, currentRadius, tenthPoint);
    }

    record Snapshot(long totalPoints, long missPoints, double currentRadius, boolean tenthPoint) {

        double area() {
            return currentRadius * currentRadius * (0.75 + Math.PI / 16.0);
        }
    }
}

```

- mbean/MonitoringRegistry.java

```

package com.example.lab3.mbean;

import javax.management.JMException;
import javax.management.MBeanServer;
import javax.management.ObjectName;
import java.lang.management.ManagementFactory;

public final class MonitoringRegistry {

    public static final String POINT_STATISTICS_OBJECT_NAME = "com.example.lab4:type=PointStatistics";
    public static final String AREA_OBJECT_NAME = "com.example.lab4:type=Area";

    private static final MonitoringState STATE = new MonitoringState();
    private static final PointStatistics POINT_STATISTICS = new PointStatistics(STATE);
    private static final Area AREA = new Area(STATE);

    private MonitoringRegistry() {
    }

    public static synchronized void registerMBeans() {
        MBeanServer server = ManagementFactory.getPlatformMBeanServer();
        try {
            register(server, new ObjectName(POINT_STATISTICS_OBJECT_NAME), POINT_STATISTICS);
            register(server, new ObjectName(AREA_OBJECT_NAME), AREA);
        } catch (JMException e) {
            throw new IllegalStateException("Unable to register lab4 MBeans", e);
        }
    }
}

```

```

public static synchronized void unregisterMBeans() {
    MBeanServer server = ManagementFactory.getPlatformMBeanServer();
    unregister(server, POINT_STATISTICS_OBJECT_NAME);
    unregister(server, AREA_OBJECT_NAME);
}

public static void registerPoint(boolean hit, double radius) {
    POINT_STATISTICS.registerPoint(hit, radius);
}

public static void updateRadius(double radius) {
    STATE.updateRadius(radius);
}

private static void register(MBeanServer server, ObjectName objectName, Object mbean) throws JMException {
    if (server.isRegistered(objectName)) {
        server.unregisterMBean(objectName);
    }
    server.registerMBean(mbean, objectName);
}

private static void unregister(MBeanServer server, String objectName) {
    try {
        ObjectName name = new ObjectName(objectName);
        if (server.isRegistered(name)) {
            server.unregisterMBean(name);
        }
    } catch (JMException ignored) {
        // Ошибка очистки MBean не должна мешать запуску приложения.
    }
}
}

```

• mbean/MBeanRegistrationListener.java

```

package com.example.lab3.mbean;

import jakarta.servlet.ServletContextEvent;
import jakarta.servlet.ServletContextListener;
import jakarta.servlet.annotation.WebListener;

@WebListener
public class MBeanRegistrationListener implements ServletContextListener {

    @Override
    public void contextInitialized(ServletContextEvent sce) {
        MonitoringRegistry.registerMBeans();
        sce.getServletContext().log("Lab4 monitoring MBeans registered");
    }

    @Override
    public void contextDestroyed(ServletContextEvent sce) {
        MonitoringRegistry.unregisterMBeans();
        sce.getServletContext().log("Lab4 monitoring MBeans unregistered");
    }
}

```

Регистрация новой точки выполняется после сохранения результата в базе данных вызовом `MonitoringRegistry.registerPoint(result.isHit(), result.getR())`. При изменении радиуса вызывается `MonitoringRegistry.updateRadius(r)`.

Площадь области вычисляется как сумма площадей прямоугольника, четверти круга и прямоугольного треугольника:

$$S = \frac{r^2}{2} + \frac{\pi r^2}{16} + \frac{r^2}{4} = r^2 \left(0,75 + \frac{\pi}{16} \right).$$

2. JConsole

Приложение развёрнуто на WildFly. Доступ к интерфейсу управления выполнялся через SSH-туннель, а подключение к JMX — по адресу `service:jmx:remote+http://127.0.0.1:10990`. В дереве MBeans зарегистрированы объекты `com.example.lab4:type=PointStatistics` и `com.example.lab4:type=Area`.

- Метаданные разработанных MBean.

The screenshot shows the JConsole interface with the 'MBeans' tab selected. The left sidebar displays a tree of MBeans, with 'Area' under 'com.example.lab4' selected. The main panel shows the metadata for the selected MBean.

Name	Value
Info:	
ObjectName	com.example.lab4:type=Area
ClassName	com.example.lab3.mbean.Area
Description	Information on the management interface of the MBean

Name	Value
Descriptor:	
immutableInfo	true
interfaceClassName	com.example.lab3.mbean.AreaMBean
mxbean	false

Рис. 1: Метаданные MBean Area в JConsole

The screenshot shows the JConsole interface with the 'MBeans' tab selected. The left sidebar displays a tree of MBeans, with 'PointStatistics' under 'com.example.lab4' selected. The main panel shows the metadata for the selected MBean.

Name	Value
Info:	
ObjectName	com.example.lab4:type=PointStatistics
ClassName	com.example.lab3.mbean.PointStatistics
Description	Information on the management interface of the MBean

Name	Value
Descriptor:	
immutableInfo	false
interfaceClassName	com.example.lab3.mbean.PointStatisticsMBean
mxbean	false

Рис. 2: Метаданные MBean PointStatistics в JConsole

- Показания разработанных MBean.

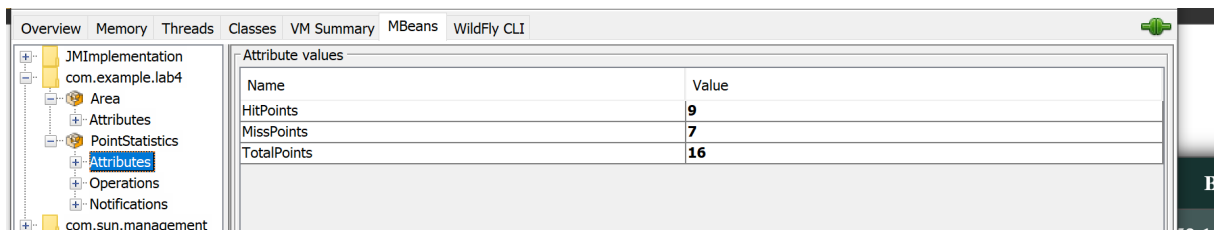


Рис. 3: Атрибуты MBean PointStatistics в JConsole

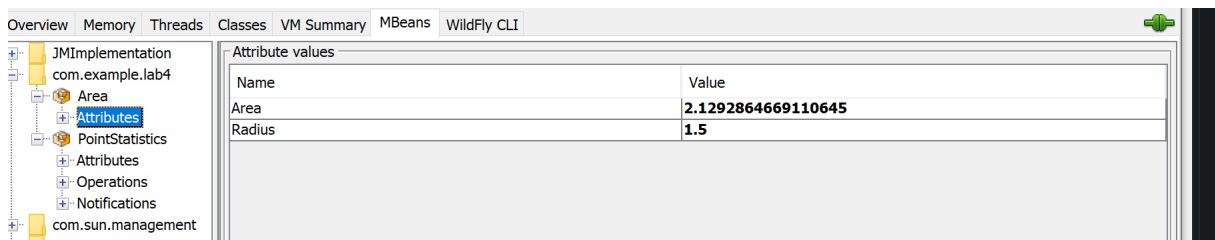


Рис. 4: Атрибуты MBean Area в JConsole

- Время работы виртуальной машины.

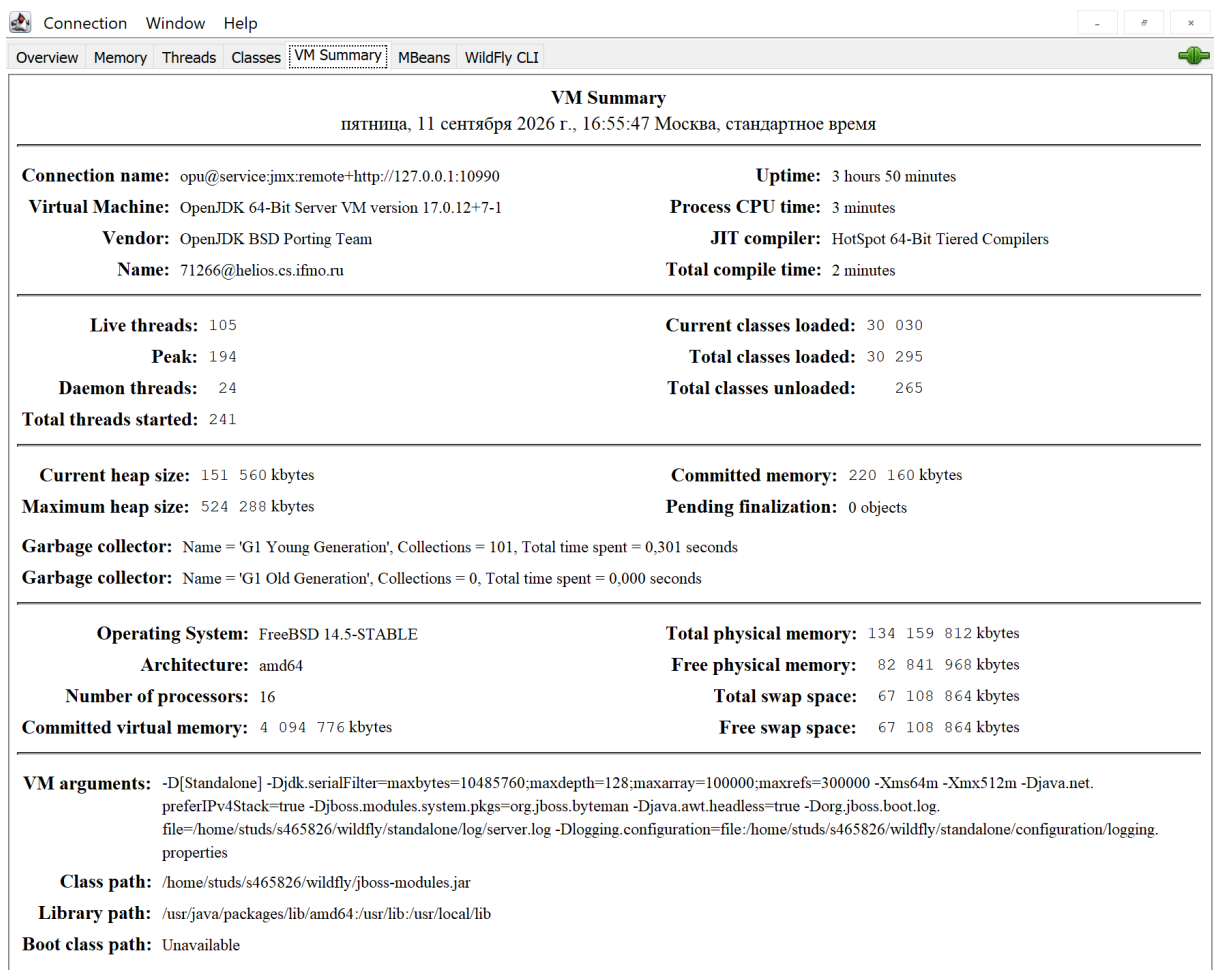


Рис. 5: Сводная информация о виртуальной машине в JConsole

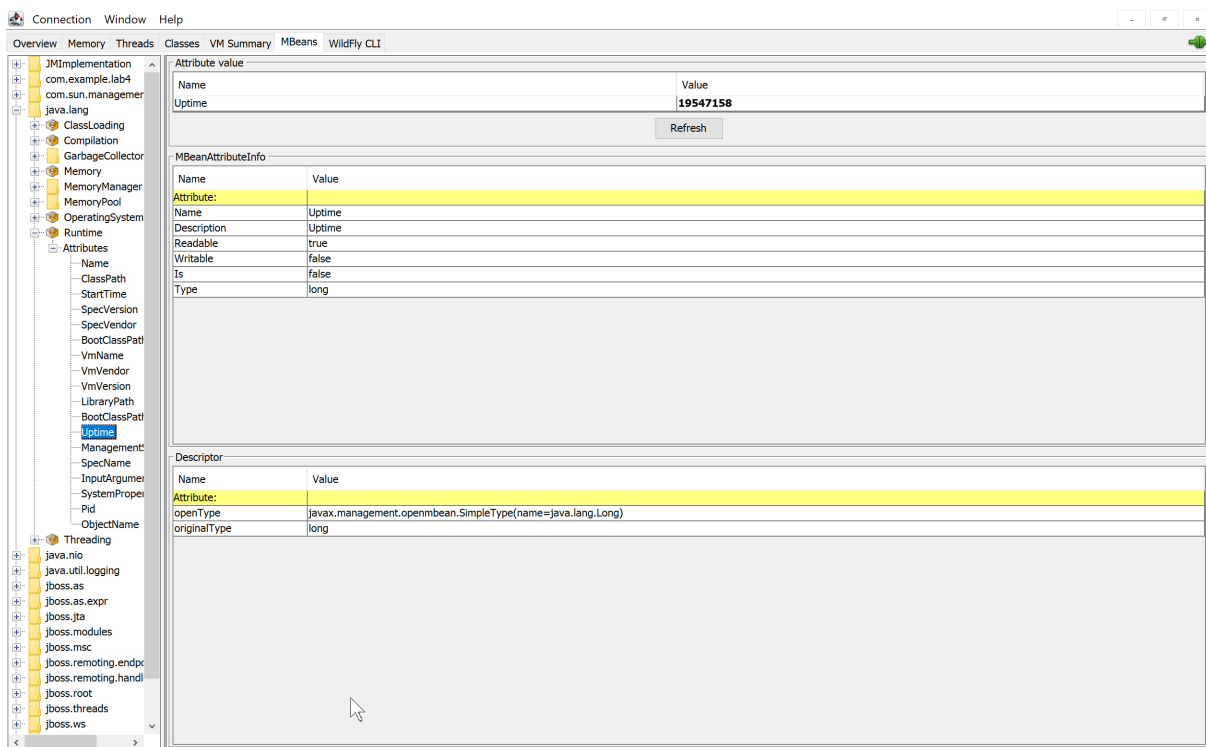


Рис. 6: Значение Uptime стандартного RuntimeMXBean в миллисекундах

- Получение уведомления.

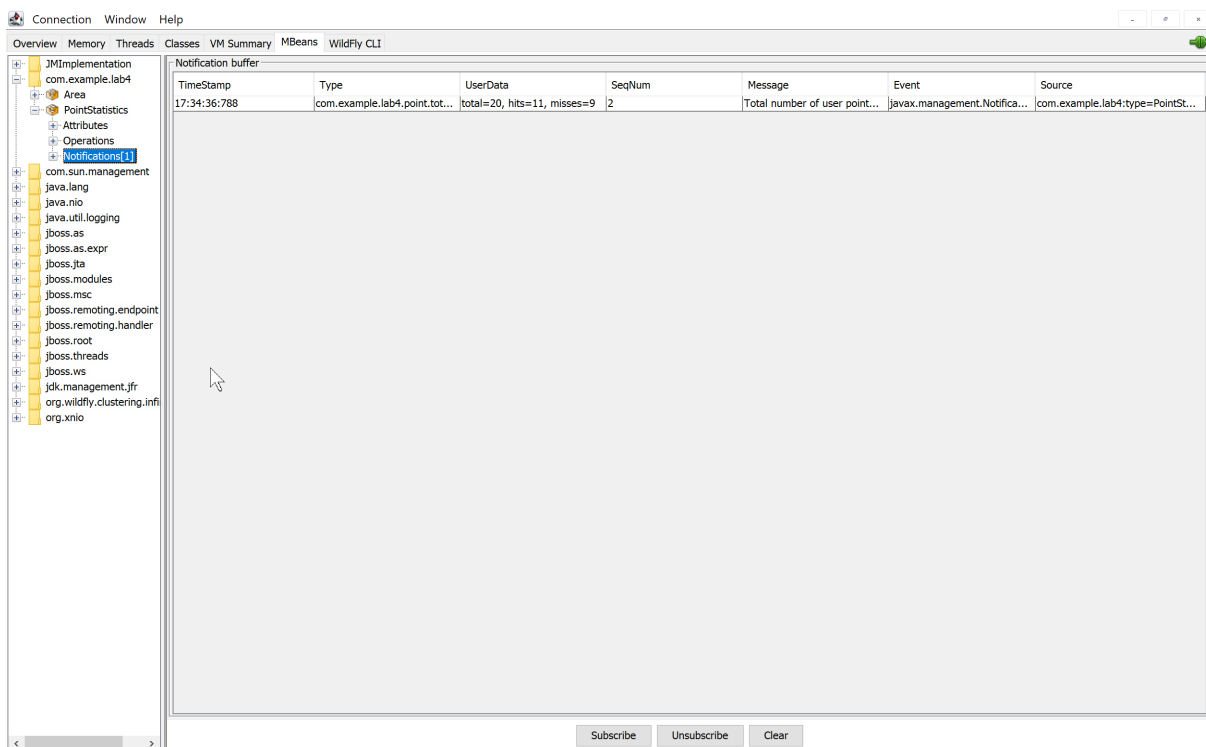


Рис. 7: JMX-уведомление при достижении количества точек, кратного 10

- Графики изменения показаний MBean.

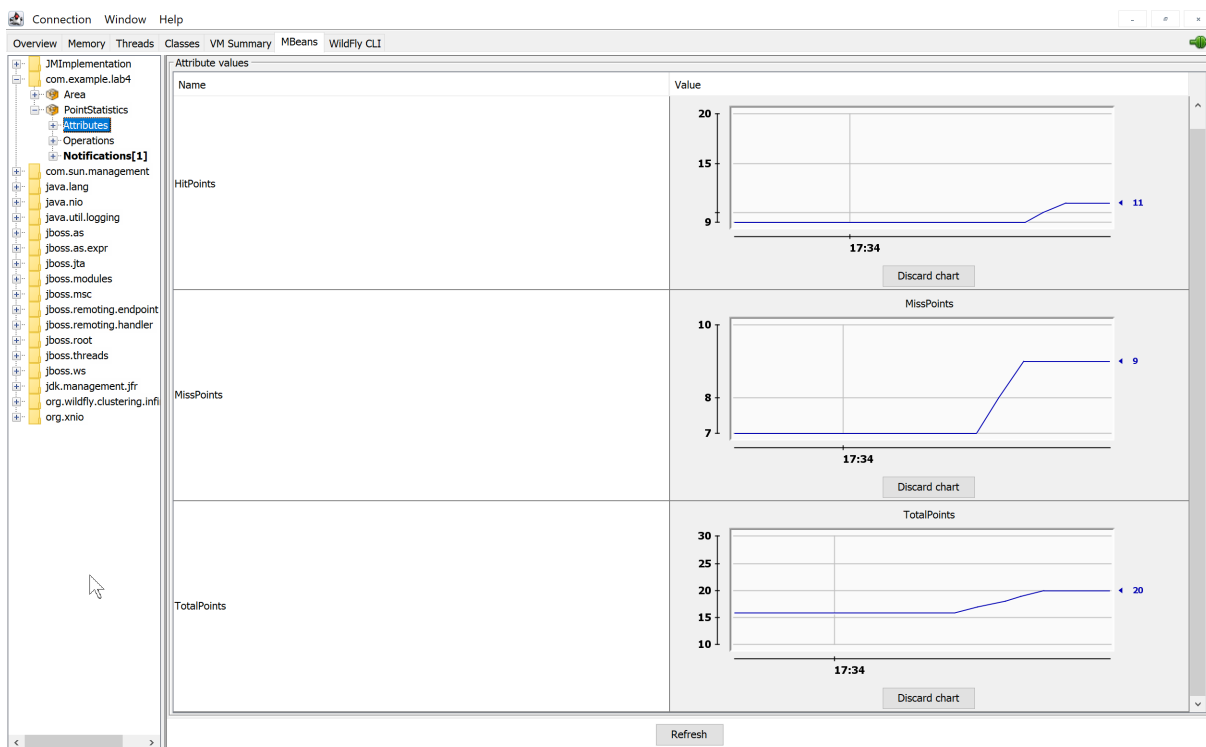


Рис. 8: Графики атрибутов PointStatistics в JConsole

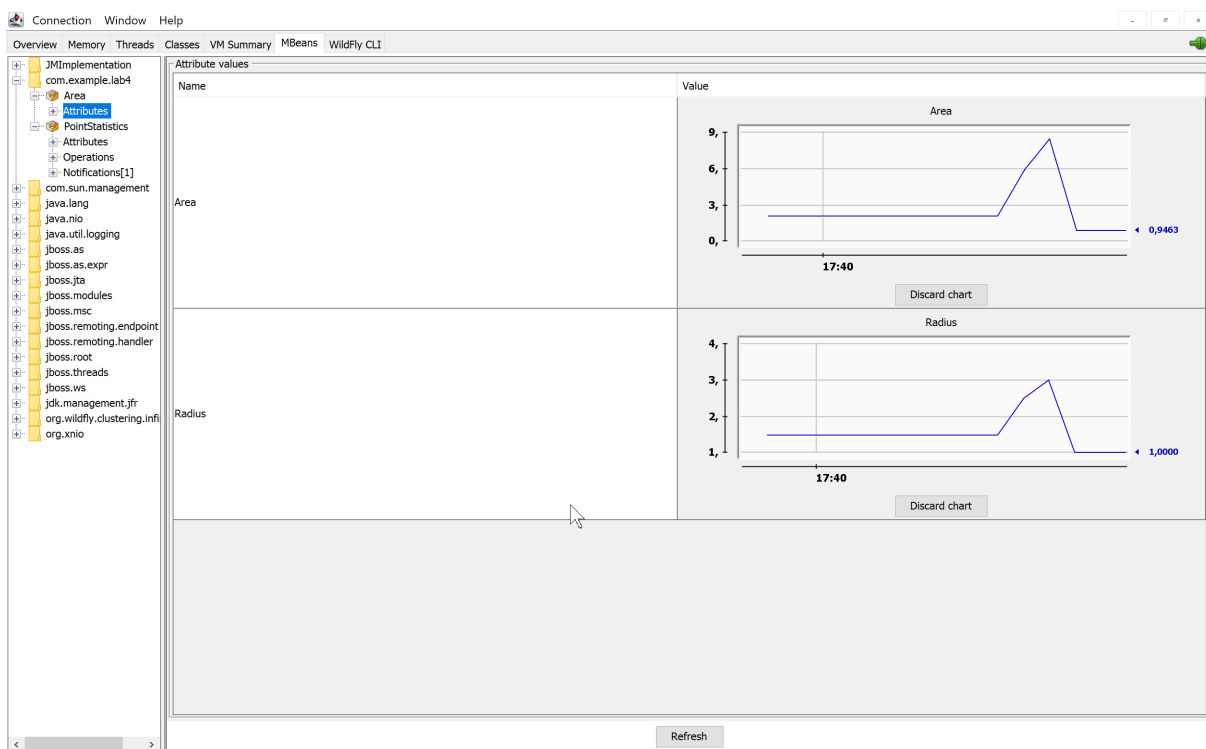


Рис. 9: Графики атрибутов Radius и Area в JConsole

- **Выводы.**

В JConsole проверяется соответствие атрибутов TotalPoints, HitPoints и MissPoints действиям пользователя. Для каждой выборки должно выполняться равенство $TotalPoints = HitPoints$

+ MissPoints. При переходе общего счётчика к значению, кратному 10, MBean PointStatistics отправляет уведомление типа `com.example.lab4.point.totalMultipleOfTen`. Время после запуска JVM определяется по атрибуту Uptime объекта `java.lang:type=Runtime`; на момент измерения оно составило 19547158 мс.

3. VisualVM

VisualVM подключён к той же JVM WildFly через встроенный JMX-интерфейс сервера. Для просмотра пользовательских MBean установлен модуль MBeans.

- Обзор JVM.

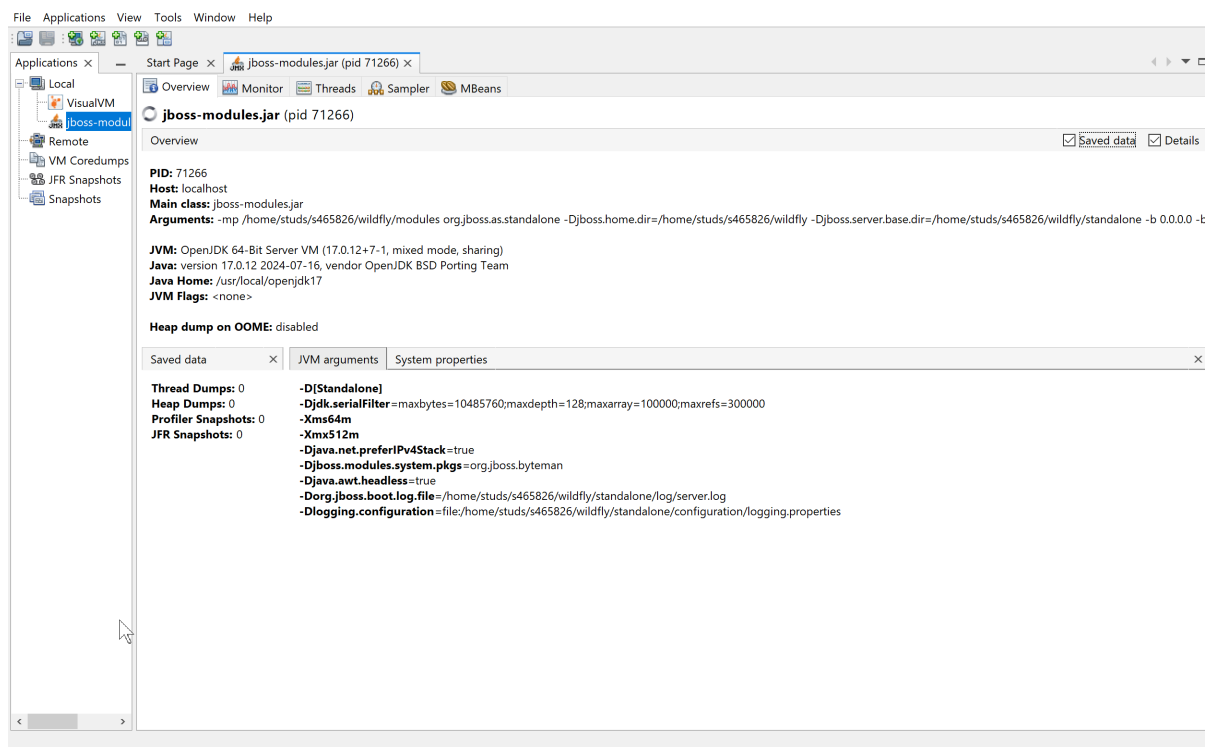


Рис. 10: Общая информация о JVM WildFly в VisualVM

- Показания разработанных MBean и их изменение во времени.

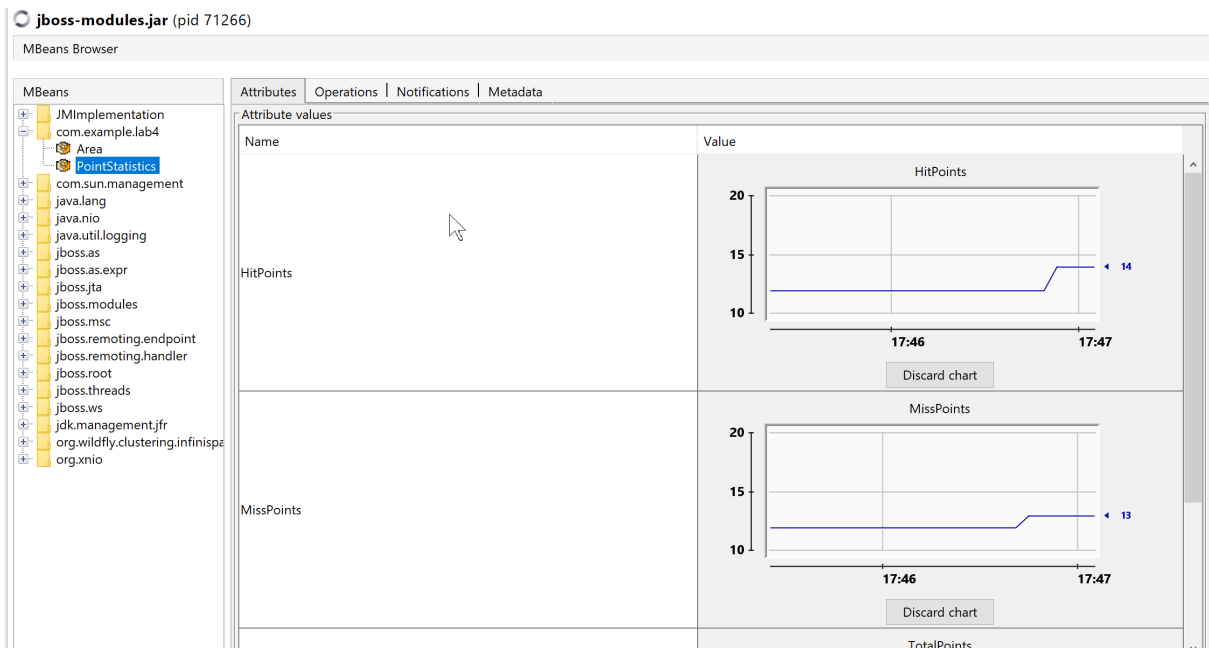


Рис. 11: Графики изменения атрибутов MBean PointStatistics в VisualVM (часть 1)

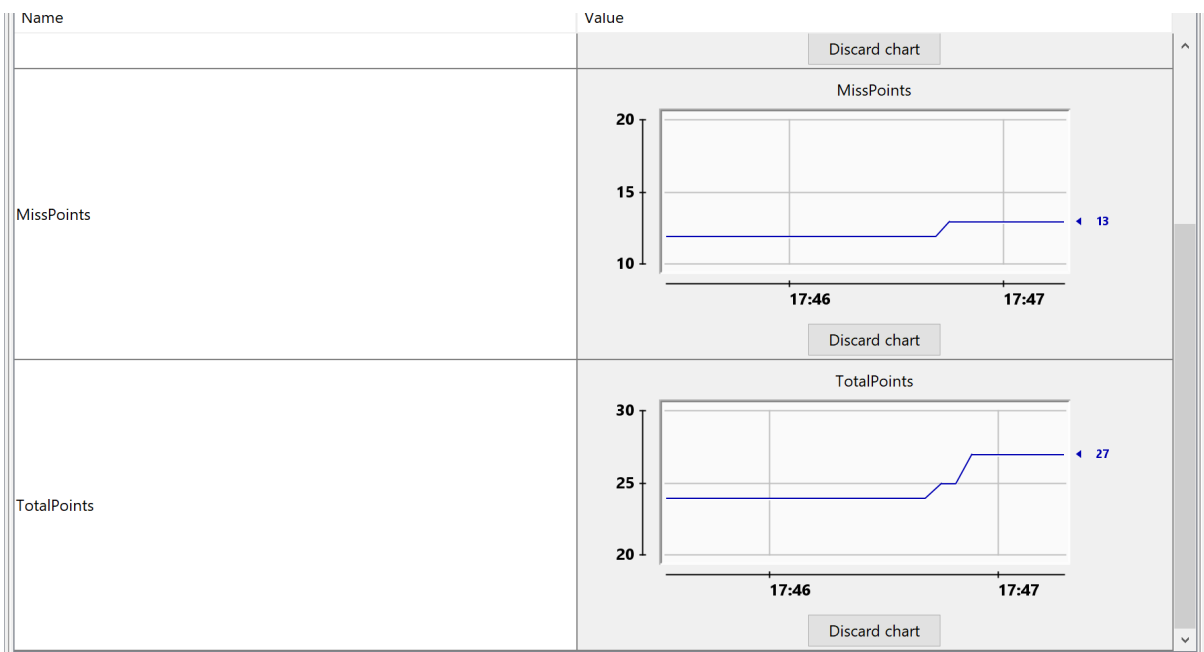


Рис. 12: Графики изменения атрибутов MBean PointStatistics в VisualVM (часть 2)

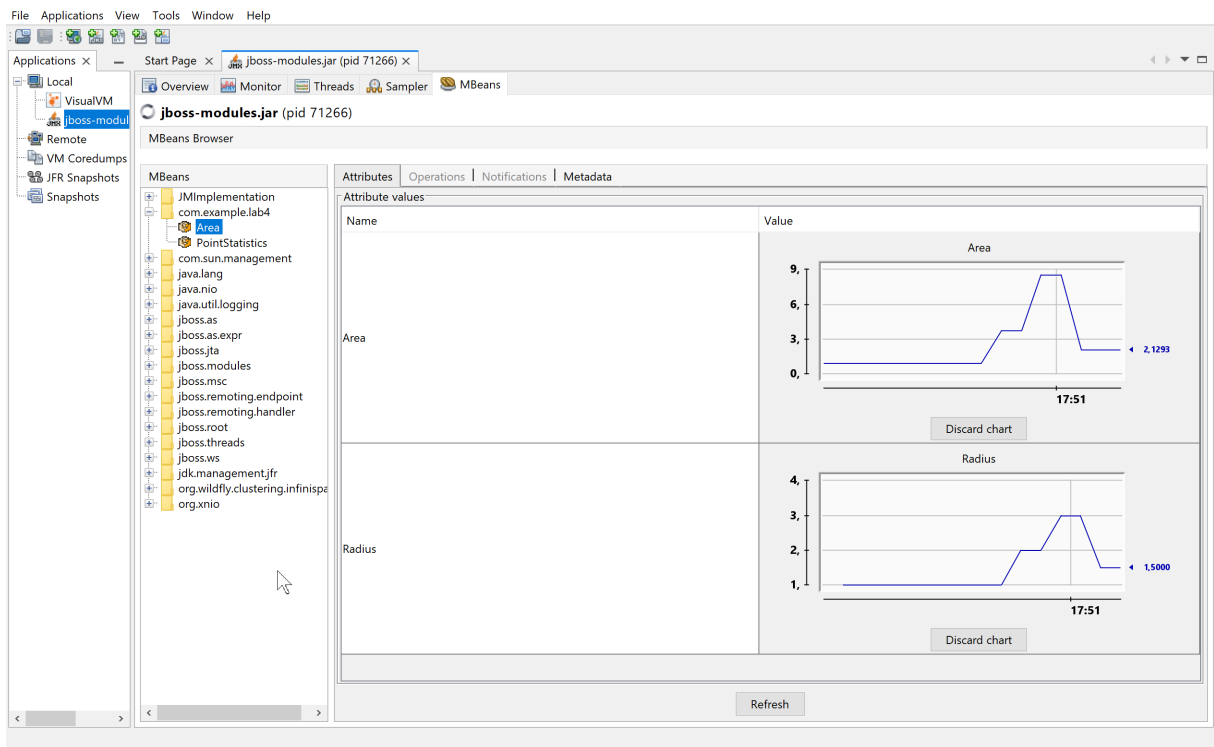


Рис. 13: Графики изменения атрибутов Radius и Area в VisualVM

- Классы, занимающие память JVM.

The screenshot shows the VisualVM Sampler window for the application `jboss-modules.jar` (pid 71266). The 'Memory' tab is selected, and the 'Status' is 'memory sampling in progress'. The 'Heap histogram' tab is active, displaying a table of memory usage statistics for various Java classes. The table has columns for 'Name', 'Live Bytes', and 'Live Objects'. The data is sorted by 'Live Bytes' in descending order.

Name	Live Bytes	Live Objects
<code>byte[]</code>	26 204 352 B (31,2 %)	263 996 (14,8 %)
<code>java.util.HashMap\$Node</code>	6 807 616 B (8,1 %)	212 738 (12 %)
<code>java.lang.String</code>	5 626 392 B (6,7 %)	234 433 (13,2 %)
<code>java.lang.Object[]</code>	3 892 712 B (4,6 %)	74 476 (4,2 %)
<code>java.lang.Class</code>	3 742 208 B (4,5 %)	31 517 (1,8 %)
<code>java.util.HashMap\$Node[]</code>	2 699 432 B (3,2 %)	19 694 (1,1 %)
<code>int[]</code>	2 647 896 B (3,2 %)	12 469 (0,7 %)
<code>java.lang.reflect.Method</code>	1 874 840 B (2,2 %)	21 305 (1,2 %)
<code>java.util.HashMap</code>	1 562 976 B (1,9 %)	32 562 (1,8 %)
<code>java.lang.reflect.Parameter</code>	1 452 320 B (1,7 %)	36 308 (2 %)
<code>java.util.concurrent.ConcurrentHashMap\$Node</code>	1 220 320 B (1,5 %)	38 135 (2,1 %)
<code>org.jboss.weld.annotated.slim.backed.BackedAnnotatedParameter</code>	1 150 816 B (1,4 %)	35 963 (2 %)
<code>java.util.ArrayList</code>	1 052 208 B (1,3 %)	43 842 (2,5 %)
<code>java.util.LinkedHashMap\$Entry</code>	585 440 B (0,7 %)	14 636 (0,8 %)
<code>org.jboss.jandex.MethodInternal</code>	500 256 B (0,6 %)	10 422 (0,6 %)
<code>org.jboss.weld.annotated.slim.backed.BackedAnnotatedMethod</code>	473 728 B (0,6 %)	14 804 (0,8 %)
<code>java.util.concurrent.ConcurrentHashMap\$Node[]</code>	471 400 B (0,6 %)	1 953 (0,1 %)
<code>java.util.Hashtable\$Entry</code>	462 912 B (0,6 %)	14 466 (0,8 %)
<code>sun.reflect.generics.tree.SimpleClassTypeSignature</code>	439 032 B (0,5 %)	18 293 (1 %)
<code>java.util.jar.JarFile\$JarFileEntry</code>	425 360 B (0,5 %)	4 090 (0,2 %)
<code>java.util.LinkedHashMap</code>	422 968 B (0,5 %)	7 553 (0,4 %)
<code>java.lang.reflect.Parameter[]</code>	413 376 B (0,5 %)	15 369 (0,9 %)

Рис. 14: Результаты профилирования памяти в VisualVM

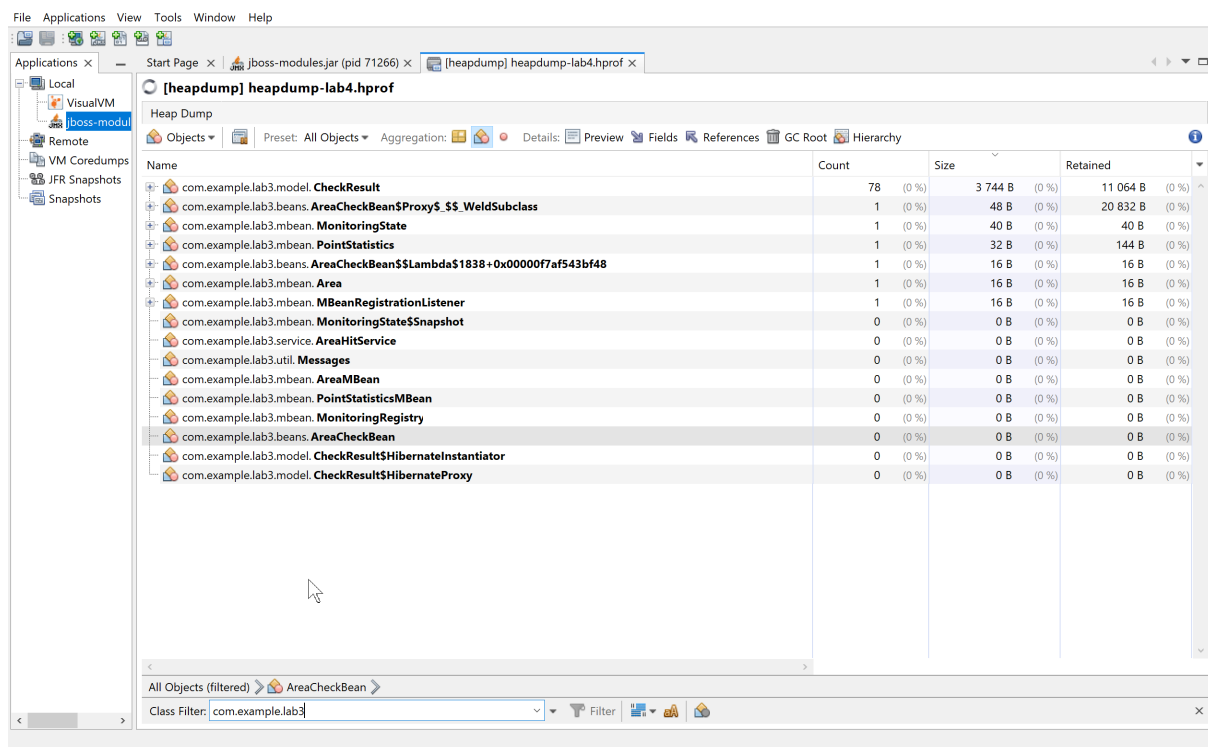


Рис. 15: Распределение памяти между классами приложения

- Выводы по результатам мониторинга и профилирования.

При добавлении точек значения **TotalPoints**, **HitPoints** и **MissPoints** изменяются в соответствии с результатами проверки. При выборе другого радиуса изменяются атрибуты **Radius** и **Area**. В общей гистограмме JVM наибольший объём памяти занимают массивы **byte[]**. После фильтрации Heap Dump по пакету приложения установлено, что среди пользовательских классов наибольший суммарный shallow size имеет **CheckResult**: 78 экземпляров занимают 3744 В и удерживают 11064 В. Единственный экземпляр прокси **AreaCheckBean** удерживает 20832 В.

4. Исследование программы на утечки памяти

Для воспроизводимого исследования необходимо зафиксировать исходное состояние памяти, создать нагрузку большим количеством проверок точек и повторить измерение после принудительного запуска сборщика мусора. Сравниваются число экземпляров, shallow size и retained size классов, продолжающих удерживаться после GC.

Перед анализом Heap Dump была зафиксирована общая динамика JVM во вкладке Monitor. Пилообразное изменение занятой памяти соответствует циклу выделения объектов и их последующего удаления сборщиком мусора. Такой график показывает активную работу GC, однако сам по себе не определяет объекты, которые продолжают удерживаться, поэтому дальнейшая локализация выполнялась по Heap Dump.

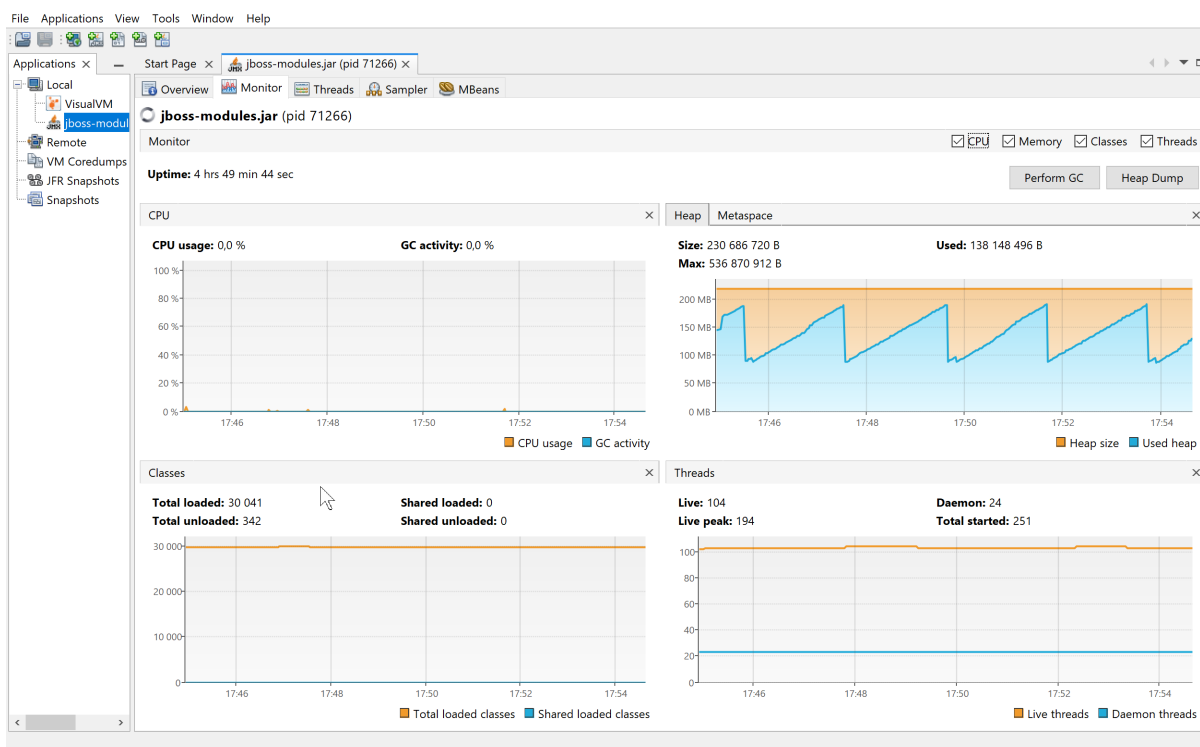


Рис. 16: Мониторинг CPU, Heap, Classes и Threads до локализации проблемы с памятью

1. Подключиться к JVM приложения в VisualVM и открыть вкладку Sampler.
2. Запустить профилирование памяти и сохранить исходный снимок.
3. Выполнить серию однотипных действий в приложении, увеличивающих историю результатов.
4. Нажать **Perform GC**, повторить снимок и отсортировать классы по занимаемой памяти.

5. Для растущего класса открыть цепочку удерживающих ссылок и определить пользовательский объект-владелец.
6. Найти соответствующее поле или коллекцию в исходном коде.
7. Внести ограничение роста коллекции либо заменить полную загрузку истории страничной выборкой.
8. Повторить тот же сценарий нагрузки и сравнить результаты до и после исправления.

До исправления объект `AreaCheckBean` с областью видимости `SessionScoped` хранил поле `List<CheckResult> results`. При инициализации в него загружалась вся история из базы данных, а после каждой проверки новый объект добавлялся в начало списка без удаления старых записей. Heap Dump подтвердил эту причину: поле `results` ссылалось на `ArrayList` из 78 элементов с retained size 11544 B.

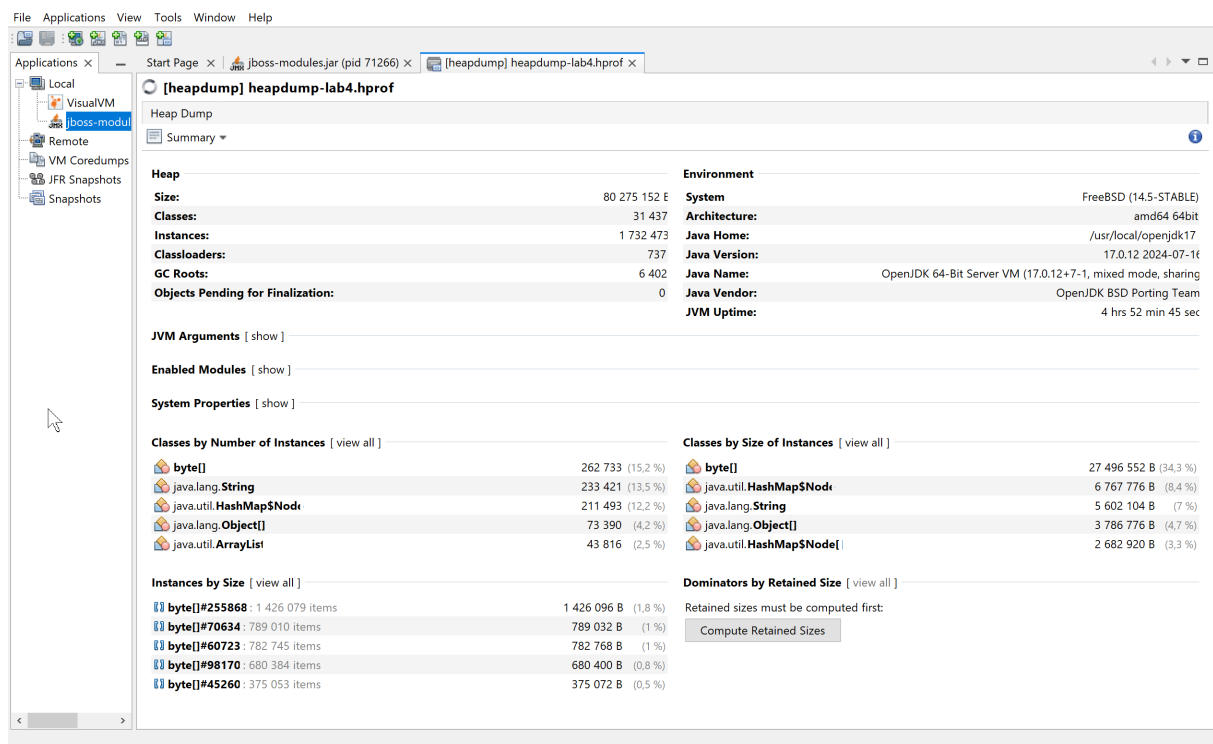


Рис. 17: Использование памяти и удерживаемые объекты до исправления

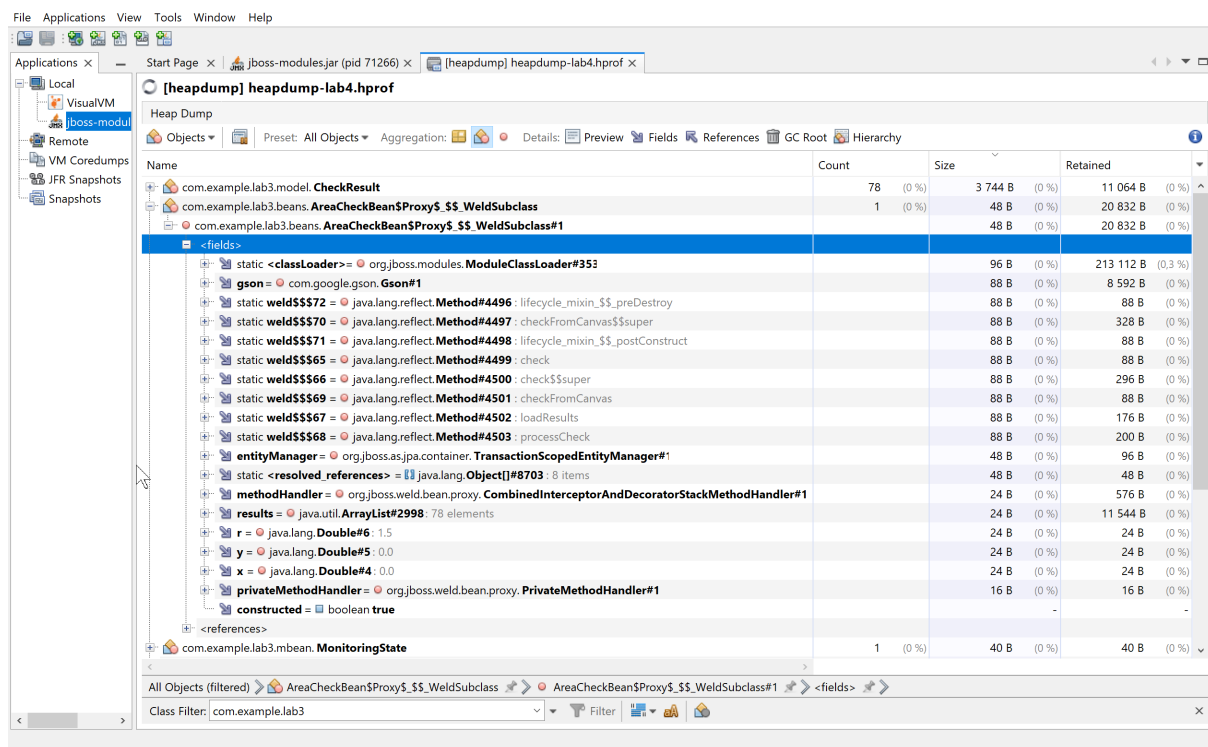


Рис. 18: Цепочка удержания объектов, позволившая локализовать проблему

```

private List<CheckResult> results;

private final transient Gson gson = new GsonBuilder()
    .registerTypeAdapter(LocalDate.class, (JsonSerializer<LocalDate>) (src, typeOfSrc, context) ->
        new JsonPrimitive(src.toString()))
    .create();

@PostConstruct
public void init() {
    loadResults();
}

@Transactional
public void check() {
    processCheck(this.x, this.y);
}

@Transactional
public void checkFromCanvas() {
    Map<String, String> params = FacesContext.getCurrentInstance().getExternalContext().getRequestParameterMap();
    try {
        double canvasX = Double.parseDouble(params.get(REQUEST_PARAM_X_KEY));
        double canvasY = Double.parseDouble(params.get(REQUEST_PARAM_Y_KEY));
        processCheck(canvasX, canvasY);
    } catch (NumberFormatException | NullPointerException e) {
        System.err.println(Messages.get("error.canvas.parse.prefix") + e.getMessage());
    }
}

private void processCheck(double x, double y) {
    CheckResult result = new CheckResult();
    result.setX(x);
    result.setY(y);
    result.setR(this.r);
    result.setHit(AreaHitService.isHit(x, y, this.r));
    result.setRequestTime(LocalDate.now());

    entityManager.persist(result);
    MonitoringRegistry.registerPoint(result.isHit(), result.getR());
    if (results != null) {
        results.add(0, result);
    }
}

private void loadResults() {
    try {
        this.results = entityManager.createQuery(RESULTS_QUERY, CheckResult.class)
            .getResultList();
    } catch (Exception e) {
        this.results = Collections.emptyList();
        System.err.println(Messages.get("error.results.load.prefix") + e.getMessage());
    }
}

```

Рис. 19: Участок исходного кода, вызывающий накопление объектов

Name	Count	Size	Retained
com.example.lab3.beans.AreaCheckBean\$Proxy\$\$_WeldSubclass	1 (0 %)	48 B (0 %)	11 400 B (0 %)
com.example.lab3.model.CheckResult	50 (0 %)	2 400 B (0 %)	7 200 B (0 %)
com.example.lab3.mbean.PointStatistics	1 (0 %)	32 B (0 %)	144 B (0 %)
com.example.lab3.mbean.PointStatistics	1 (0 %)	32 B (0 %)	112 B (0 %)
com.example.lab3.mbean.MonitoringState	1 (0 %)	40 B (0 %)	40 B (0 %)
com.example.lab3.mbean.MonitoringState	1 (0 %)	40 B (0 %)	40 B (0 %)
com.example.lab3.beans.AreaCheckBean\$\$Lambda\$2069+0x00000f7af56d8000	1 (0 %)	16 B (0 %)	16 B (0 %)
com.example.lab3.mbean.Area	1 (0 %)	16 B (0 %)	16 B (0 %)
com.example.lab3.mbean.MBeanRegistrationListener	1 (0 %)	16 B (0 %)	16 B (0 %)
com.example.lab3.beans.AreaCheckBean\$\$Lambda\$1838+0x00000f7af543bf48	1 (0 %)	16 B (0 %)	16 B (0 %)
com.example.lab3.mbean.Area	1 (0 %)	16 B (0 %)	16 B (0 %)
com.example.lab3.mbean.MBeanRegistrationListener	1 (0 %)	16 B (0 %)	16 B (0 %)
com.example.lab3.service.AreaHitService	0 (0 %)	0 B (0 %)	0 B (0 %)
com.example.lab3.util.Messages	0 (0 %)	0 B (0 %)	0 B (0 %)
com.example.lab3.mbean.MonitoringState\$Snapshot	0 (0 %)	0 B (0 %)	0 B (0 %)
com.example.lab3.mbean.AreaMBean	0 (0 %)	0 B (0 %)	0 B (0 %)
com.example.lab3.mbean.PointStatisticsMBean	0 (0 %)	0 B (0 %)	0 B (0 %)
com.example.lab3.mbean.MonitoringRegistry	0 (0 %)	0 B (0 %)	0 B (0 %)
com.example.lab3.beans.AreaCheckBean	0 (0 %)	0 B (0 %)	0 B (0 %)
com.example.lab3.model.CheckResult\$HibernateInstantiator	0 (0 %)	0 B (0 %)	0 B (0 %)
com.example.lab3.model.CheckResult\$HibernateProxy	0 (0 %)	0 B (0 %)	0 B (0 %)
com.example.lab3.mbean.MonitoringState\$Snapshot	0 (0 %)	0 B (0 %)	0 B (0 %)
com.example.lab3.service.AreaHitService	0 (0 %)	0 B (0 %)	0 B (0 %)
com.example.lab3.util.Messages	0 (0 %)	0 B (0 %)	0 B (0 %)
com.example.lab3.mbean.AreaMBean	0 (0 %)	0 B (0 %)	0 B (0 %)

Рис. 20: Результат повторного профилирования после исправления

Для устранения неограниченного роста запрос к базе данных ограничен 50 последними результатами, а после добавления новой записи последний элемент удаляется, если размер списка превысил 50. После повторного развёртывания приложения, аналогичной нагрузки и принудительного запуска сборщика мусора в Heap Dump осталось 50 экземпляров CheckResult. Их shallow size уменьшился с 3744 В до 2400 В, а retained size — с 11064 В до 7200 В. Retained size объекта AreaCheckBean уменьшился с 20832 В до 11400 В. Таким образом, число удерживаемых результатов и их размер сократились примерно на 36%, а дальнейший рост списка ограничен.

Вывод

В ходе лабораторной работы были реализованы и зарегистрированы MBean-классы для наблюдения за количеством проверенных точек, количеством промахов и площадью области. С помощью JConsole проверены значения атрибутов, время работы JVM и получение уведомления после каждой десятой точки. В VisualVM исследованы изменения показателей во времени и распределение памяти JVM. Heap Dump позволил обнаружить неограниченное накопление объектов `CheckResult` в сессионном объекте `AreaCheckBean`. Ограничение истории 50 последними результатами устранило дальнейший рост коллекции, что подтверждено повторным профилированием.